专升本 **Java** 程序设计模拟卷-卷 **2**

fifine-专升本

2026 年 3 月 2 日

fifine专升本

一、单项选择题深度解析（共 20 小题）

1. Java 程序的执行过程是由解释器执行编译后生成的字节码文件，该字节码文件的扩展名是（　）。

a) .java　　b) .class　　c) .exe　　d) .jar

答案与深度解析 答案：**B. .class**

【核心认知框架】

- 题目本质：考查 Java 编译体系结构及文件产物规范
- 关键区分点：源码 (.java)、字节码 (.class)、可执行文件 (.exe)、归档文件 (.jar)
- 命题人意图：测试对 Java 跨平台核心机制（Write Once, Run Anywhere）基础载体的认知

【选项原理深度拆解】

- **A. .java** —错误（源代码文件）
  - 字节码铁证：

```
// Hello.java
public class Hello { }
// javac Hello.java 生成 Hello.class
// .java 文件无法直接被 JVM 加载
```

    JVM 规范依据：JVMS §4 规定 JVM 加载的是 Class 文件格式，而非 Java 源文件格式
  - 编译阶段：javac 编译器读取.java 文件，输出.class 文件
- **B. .class** —正确（字节码文件）
  - 文件结构：遵循 JVMS §4 定义的魔数 (0xCAFEBABE)、版本号、常量池等结构
  - 执行机制：JVM 类加载器 (ClassLoader) 专门识别并加载.class 文件
  - 跨平台性：.class 文件是平台无关的二进制格式，依赖特定平台的 JVM 解释/编译执行
- **C. .exe** —错误（**Windows** 可执行文件）
  - 本质：Windows 平台特有的本机代码 executable
  - **Java** 关系：Java 程序可通过 jlink 或第三方工具打包为 exe，但核心字节码仍是.class
- **D. .jar** —错误（归档文件）
  - 本质：基于 ZIP 格式的归档文件，内部包含.class 文件及 MANIFEST.MF
  - 用途：用于分发和部署，JVM 加载的是 JAR 内部的.class 文件

【应背诵知识点（命题人思维版）】

fffine专升本

(a) **Java** 编译流程：.java (源) → javac → .class (字节码) → JVM → 机器码
(b) 魔数识别：所有.class 文件前 4 字节为 0xCAFEBABE
【命题趋势与实战策略】
• 考场秒杀：见“字节码”选.class，见“源码”选.java

2. 在 **Java** 中，下列哪个标识符是不合法的？（　）

a) _studentName　　b) $salary　　c) 2ndNumber　　d) MAX_VALUE

答案与深度解析 答案：**C. 2ndNumber**
【核心认知框架】
• 题目本质：考查 JLS §3.8 标识符词法规范
• 关键区分点：首字符不能为数字，允许 _ 和 $
• 命题人意图：测试对变量命名规则的死记硬背与理解
【选项原理深度拆解】
• **A. _studentName** —正确（合法）
  – 规范：下划线 _ 是合法的首字符
  – 字节码：编译通过，常量池中保留原名
• **B. $salary** —正确（合法）
  – 规范：美元符号 `$` 是合法的首字符（常用于内部类）
  – 示例：`Outer$Inner.class`
• **C. 2ndNumber** —错误（非法）
  – 规范：JLS §3.8 规定标识符不能以数字开头
  – 编译错误：`int 2ndNumber = 1;` → ”Illegal start of type”
  – 词法分析：编译器将 2 识别为数字字面量，后续 `ndNumber` 无法衔接
• **D. MAX_VALUE** —正确（合法）
  – 规范：符合命名规范，常用于常量
  – 示例：`Integer.MAX_VALUE`
【应背诵知识点（命题人思维版）】
• 标识符组成：字母、数字、_、$
• 限制：数字不能开头，不能是关键字

fifine专升本

3. Java 基本数据类型中，`char` 类型所占用的内存空间是（　）个字节。

a) 1　　b) 2　　c) 4　　d) 8

答案与深度解析 答案：**B. 2**【核心认知框架】：考查 JVM 基本类型内存布局 (JVMS §2.3.2)。【深度解析】：

- 规范：Java `char` 是 16-bit Unicode 字符，固定占用 2 字节。
- 对比：C/C++ `char` 通常为 1 字节，Java 为国际化设计。
- 字节码：`char` 在栈帧中占用 1 个 Slot (32-bit)，但有效数据为 16-bit。

【应背诵】：char=2 字节，boolean 虚拟机规范未明确定义（通常 1 字节或 4 字节）。

4. 假设 `a` 为整型变量，初值为 **5**，则执行表达式 `a += a * 2` 后，`a` 的值为（　）。

a) 10　　b) 15　　c) 20　　d) 25

答案与深度解析 答案：**B. 15**【核心认知框架】：考查运算符优先级及复合赋值运算。【深度解析】：

- 优先级：`*` 高于 `+=`。
- 计算过程：`a * 2` → `5 * 2 = 10`；`a += 10` → `5 + 10 = 15`。
- 字节码：`iload`, `iconst_2`, `imul`, `iadd`, `istore`。

5. 下列哪种数据类型不能作为 `switch` 语句的条件测试变量？（　）

a) byte　　b) long　　c) char　　d) String

答案与深度解析 答案：**B. long**【核心认知框架】：考查 switch 支持的数据类型演变。【深度解析】：

- 支持类型：byte, short, char, int, enum, String (JDK7+), 包装类 (JDK5+)。
- 不支持：long, float, double, boolean。
- 原因：switch 底层基于跳转表 (lookupswitch/tableswitch)，long 范围过大不适合。

6. 在定义二维数组时，下列哪种写法是错误的？（　）

a) `int[][] a = new int[3][4];`　b) `int a[][] = new int[3][];`　c) `int[][] a = new int[][4];`　d) `int[] a[] = {{1,2},{3,4}};`

答案与深度解析 答案：**C. `int[][] a = new int[][4];`**【核心认知框架】：考查数组初始化规则。【深度解析】：

- 规则：Java 多维数组是“数组的数组”，第一维长度必须指定，后续可省略。
- 错误原因：`new int[][4]` 缺少第一维长度，编译器无法分配内存。
- 正确写法：`new int[3][4]` 或 `new int[3][]`。

fifine专升本

7. 面向对象程序设计的三个主要特征不包括（　）。

a) 封装性　　b) 继承性　　c) 健壮性　　d) 多态性

答案与深度解析 答案：**C.** 健壮性 【核心认知框架】：考查 OOP 三大支柱。【深度解析】：
- 三大特征：封装 (Encapsulation)、继承 (Inheritance)、多态 (Polymorphism)。
- 健壮性：是编程语言的目标或特性，非 OOP 核心定义特征。

8. 关于 **Java** 类中的构造方法，下列说法正确的是（　）。

a) 构造方法的方法名可以和类名不同　　b) 构造方法必须声明返回值类型,即使是 `void`　　c) 一个类只能有一个构造方法　　d) 构造方法在通过 `new` 关键字实例化对象时自动调用

答案与深度解析 答案：**D.** 构造方法在通过 `new` 关键字实例化对象时自动调用【核心认知框架】：考查构造方法语法规则。【深度解析】：
- **A** 错：构造方法名必须与类名完全相同。
- **B** 错：构造方法不能有返回值类型（包括 void），否则变为普通方法。
- **C** 错：支持重载，可以有多个构造方法。
- **D** 对：`new` 指令触发构造方法调用 (`invokespecial`)。

fifine专升本

9. 若要在子类的构造方法中调用父类的有参构造方法，必须使用（　）关键字，并且必须放在子类构造方法的第一行。

a) this　　b) super　　c) final　　d) static

答案与深度解析 答案：**B. super** 【核心认知框架】：考查继承链初始化顺序。【深度解析】：
- 机制：`super(...)` 调用父类构造器。
- 限制：必须在第一行，确保父类先初始化。
- 默认行为：若未写，编译器默认插入 `super()`。

10. 下列关于 `final` 关键字的描述，错误的是（　）。

a) 被 `final` 修饰的类不能被继承
b) 被 `final` 修饰的方法不能被重写
c) 被 `final` 修饰的变量即为常量，一旦赋值后不能修改
d) 抽象类可以被 `final` 修饰

答案与深度解析 答案：**D.** 抽象类可以被 `final` 修饰 【核心认知框架】：考查 final 与 abstract 的语义冲突。【深度解析】：
- 冲突：`abstract` 要求被继承，`final` 禁止继承，语义矛盾。
- 编译错误：`public final abstract class A` → 报错。
- 其他选项：A、B、C 均为 final 的正确特性。

11. 在 **Java** 中，所有类的顶级父类是（　）。

a) `java.lang.Class`
b) `java.lang.Object`
c) `java.util.Scanner`
d) `java.lang.System`

答案与深度解析 答案：**B.** `java.lang.Object` 【核心认知框架】：考查类继承树根节点。【深度解析】：
- 规范：JLS §4.3.2 规定所有类隐式继承 Object。
- 方法：`toString()`, `equals()`, `hashCode()` 等均源自 Object。

12. 接口（**Interface**）中的成员变量默认的修饰符是（　）。

a) `private static final`
b) `public static final`
c) `protected static`
d) 默认无修饰符

答案与深度解析 答案：**B.** `public static final` 【核心认知框架】：考查接口字段隐式修饰符。【深度解析】：
- 规范：JLS §9.3 规定接口字段隐式为 public static final。
- 必须初始化：接口字段必须在声明时赋值。

13. 当尝试访问数组的非法索引时，**Java** 运行时会抛出下列哪个异常？（　）

a) `NullPointerException`
b) `ArithmeticException`
c) `ArrayIndexOutOfBoundsException`
d) `NumberFormatException`

答案与深度解析 答案：**C.** `ArrayIndexOutOfBoundsException` 【核心认知框架】：考查运行时异常类型。【深度解析】：
- 机制：JVM 在数组访问指令 (`iaload` 等) 前插入边界检查。

fifine专升本

- 触发：索引 < 0 或索引 >= length。

14. 下列哪个类主要用于处理频繁修改的字符串序列，且是线程安全的？（　）

a) `String`　b) `StringBuilder`　c) `StringBuffer`　d) `Character`

答案与深度解析 答案：**C. `StringBuffer`**【核心认知框架】：考查字符串类线程安全性区别。【深度解析】：
- **String**：不可变。
- **StringBuilder**：可变，非线程安全（快）。
- **StringBuffer**：可变，线程安全（方法带 synchronized，慢）。

15. **Java** 集合框架中，如果希望元素能够自动保持排序（如按自然顺序），应选择下列哪个实现类？（　）

a) `HashSet`　b) `ArrayList`　c) `LinkedList`　d) `TreeSet`

答案与深度解析 答案：**D. `TreeSet`**【核心认知框架】：考查集合排序特性。【深度解析】：
- **TreeSet**：基于 TreeMap (红黑树)，元素有序。
- **HashSet**：无序。
- **List**：有序但指插入顺序，非排序。

16. 在 **Java I/O** 流中，以字符为单位进行读写操作的基础抽象类是（　）。

a) `InputStream` 和 `OutputStream`　b) `Reader` 和 `Writer`　c) `FileInputStream` 和 `FileOutputStream`　d) `DataInputStream` 和 `DataOutputStream`

答案与深度解析 答案：**B. `Reader` 和 `Writer`**【核心认知框架】：考查 IO 流分类（字节 vs 字符）。【深度解析】：
- 字节流：InputStream/OutputStream (处理二进制)。
- 字符流：Reader/Writer (处理文本，含编码转换)。

17. 下列创建和启动多线程的语句中，正确的是（　）。

a) `new Thread(...).run();`　b) `new Thread(...).start();`　c) `new RunnableImpl().start();`　d) `Thread.start(...);`

fifine专升本

答案与深度解析 答案：**B.** `new Thread(...).start();` 【核心认知框架】：考查线程启动机制。【深度解析】：

- **start()**：启动新线程，调用 run()。
- **run()**：普通方法调用，在当前线程执行。
- **Runnable**：无 start 方法。

18. 要使某个类具备被序列化的能力，该类必须实现（　）接口。

a) `Cloneable`　　b) `Runnable`　　c) `Serializable`　　d) `Comparable`

答案与深度解析 答案：**C.** `Serializable` 【核心认知框架】：考查序列化标记接口。【深度解析】：

- 标记接口：无方法，仅作为 JVM 序列化机制的标志。
- 机制：ObjectOutputStream 检查该接口，无则抛 NotSerializableException。

19. GUI 编程中，`JFrame` 的默认布局管理器是（　）。

a) `FlowLayout`　　b) `GridLayout`　　c) `BorderLayout`　　d) `CardLayout`

答案与深度解析 答案：**C.** `BorderLayout` 【核心认知框架】：考查 Swing 容器默认布局。【深度解析】：

- **JFrame**：内容面板默认 BorderLayout (北南西中东)。
- **JPanel**：默认 FlowLayout。

20. JDBC 操作数据库时，用于执行预编译 **SQL** 语句的对象是（　）。

a) `Statement`　　b) `PreparedStatement`　　c) `CallableStatement`　　d) `ResultSet`

答案与深度解析 答案：**B.** `PreparedStatement` 【核心认知框架】：考查 JDBC 接口区别。【深度解析】：

- **PreparedStatement**：预编译，防 SQL 注入，效率高。
- **Statement**：普通执行。
- **CallableStatement**：存储过程。

fifine专升本

二、判断题深度解析（共 10 小题）

（正确的选 *A*，错误的选 *B*，深度分析命题真伪）

1. Java 中的数组一旦创建，其长度就固定不可改变。

> 答案与深度解析 答案：**A (正确)**【核心认知框架】: 考查数组对象的不可变性。【深度解析】:
> - 内存模型：数组对象在堆中分配，length 字段为 final。
> - 字节码：`arraylength` 指令读取长度，无写入指令。
> - 对比：ArrayList 是可变长度的封装，底层数组仍固定（扩容是建新数组）。

2. == 运算符用于比较两个对象的内存地址是否相同，而 **`equals()`** 方法在未被重写时比较的是对象的属性值。

> 答案与深度解析 答案：**B (错误)**【核心认知框架】: 考查 equals 默认行为。【深度解析】:
> - 真相：Object 类的 `equals()` 默认实现就是 `this == obj`，比较地址。
> - 重写：只有重写后（如 String）才比较内容。
> - 命题陷阱：混淆“默认行为”与“重写后行为”。

3. 在 **Java** 中，一个子类只能继承一个父类，但一个类可以实现多个接口。

> 答案与深度解析 答案：**A (正确)**【核心认知框架】: 考查单继承多实现机制。【深度解析】:
> - 设计原因：避免菱形继承问题（C++ 痛点）。
> - 字节码：Class 文件结构中 only one super_class，multiple interfaces 数组。

4. 父类的构造方法可以被子类继承和直接使用。

> 答案与深度解析 答案：**B (错误)**【核心认知框架】: 考查构造方法继承性。【深度解析】:
> - 规则：构造方法不可继承。
> - 调用：子类通过 `super()` 隐式或显式调用。

5. 带有 **`try-catch-finally`** 的异常处理结构中，无论是否发生异常，**`finally`** 块中的代码通常都会被执行。

fitime专升本

答案与深度解析 答案：**A (正确)** 【核心认知框架】：考查 finally 执行语义。【深度解析】：

- 例外：`System.exit(0)` 或 JVM 崩溃时不执行。
- 字节码：`jsr` (旧) 或异常表属性保证 finally 跳转。

6. `String` 类型的对象是不可变的（**Immutable**），一旦创建就不能修改其内容。

答案与深度解析 答案：**A (正确)** 【核心认知框架】：考查 String 不可变性。【深度解析】：

- 底层：char[] 或 byte[] 被 final 修饰且无修改方法。
- 优势：线程安全、哈希码缓存、字符串池安全。

7. 在多重 `catch` 块中，必须将捕获 `Exception` 类的块放在捕获 `IOException` 类的块之前。

答案与深度解析 答案：**B (错误)** 【核心认知框架】：考查 catch 块顺序规则。【深度解析】：

- 规则：子类异常必须放在父类异常之前。
- 原因：否则子类 catch 块永远无法到达（Unreachable Code）。
- 正确：`IOException` 在前，`Exception` 在后。

8. `ArrayList` 内部是基于动态数组实现的，因此它的随机访问速度比 `LinkedList` 更快。

答案与深度解析 答案：**A (正确)** 【核心认知框架】：考查集合底层数据结构性能。【深度解析】：

- **ArrayList**：O(1) 随机访问 (get)。
- **LinkedList**：O(n) 随机访问 (需遍历)。

9. 包含抽象方法的类必须声明为抽象类，但抽象类也可以不包含任何抽象方法。

答案与深度解析 答案：**A (正确)** 【核心认知框架】：考查抽象类定义规则。【深度解析】：

- 强制：有抽象方法 → 必须是抽象类。
- 允许：抽象类 → 可以无抽象方法（用于限制实例化）。

fifine专升本

10. 方法内部声明的局部变量如果没有进行显式初始化，系统会自动为其赋予默认值。

答案与深度解析 答案：**B (**错误**)**【核心认知框架】：考查局部变量初始化规则。【深度解析】：

- 规则：局部变量无默认值，使用前必须初始化。
- 对比：成员变量有默认值 (0, null, false)。
- 编译错误：`variable might not have been initialized`。

fifine专升本

三、填空题深度解析（共 5 小题）

1. **Java** 的跨平台特性主要依赖于 **Java** 虚拟机（**JVM**）。**Java** 源代码被编译后生成的中间文件称为_____，其扩展名是_____。

> 答案与深度解析 答案：字节码（或字节码文件）；**.class** 【核心认知框架】：考查 Java 编译产物。【深度解析】：
> - 机制：javac 编译源文件为字节码，JVM 解释执行。
> - 跨平台：字节码统一，JVM 适配各 OS。

2. 面向对象的三大核心特征分别是:封装、_____ 和 _____。

> 答案与深度解析 答案：继承；多态 【核心认知框架】：考查 OOP 理论基础。【深度解析】：
> - 封装：隐藏细节，暴露接口。
> - 继承：代码复用，is-a 关系。
> - 多态：同一接口不同实现，动态绑定。

3. 在 **Java** 的异常处理体系中，所有的异常类都继承自 `Throwable` 类，它主要有两个直接子类：_____（用于系统严重错误）和 _____（用于程序可以处理的异常）。

> 答案与深度解析 答案：**Error**；**Exception** 【核心认知框架】：考查异常类继承树。【深度解析】：
> - **Error**：JVM 层面错误（如 OOM），不可恢复。
> - **Exception**：程序逻辑错误，可捕获处理。

4. **Java** 中线程的生命周期包含了新建（**New**）、就绪（**Runnable**）、_____（**Running**）、_____（**Blocked**）以及死亡（**Dead**）五种基本状态。

> 答案与深度解析 答案：运行（状态）；阻塞（状态）【核心认知框架】：考查线程状态模型。【深度解析】：
> - **JVM** 规范：JVMS §2.7 定义线程状态。
> - **API** 映射：Thread.State 枚举 (NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED)。

ffine专升本

5. 在 **Java** 集合框架中，_____ 接口的实现类允许存在重复的元素并且是有序的；而 _____ 接口的实现类不允许存在重复的元素。

答案与深度解析 答案：**List**；**Set**【核心认知框架】：考查 Collection 子接口特性。【深度解析】：

- **List**：有序可重复 (ArrayList, LinkedList)。
- **Set**：无序不可重复 (HashSet, TreeSet)。

fifine专升本

四、程序运行结果题深度解析（共 5 小题）

1. 写出下列程序的输出结果：

```
public class Test1 {
public static void main(String[] args) {
int sum = 0;
for (int i = 1; i <= 5; i++) {
if (i == 3) continue;
sum += i;
}
System.out.println("sum␣=␣" + sum);
}
}
```

答案与深度解析 答案：`sum = 12` 【核心认知框架】：考查循环控制流 continue 语义。【执行流程深度拆解】：

- 逻辑：i=1,2,4,5 累加；i=3 跳过。
- 计算：1+2+4+5 = 12。
- 字节码：`continue` 对应跳转指令跳过累加步骤。

2. 写出下列程序的输出结果：

```
public class Test2 {
public static void change(int[] arr) {
arr[0] = 99;
}
public static void main(String[] args) {
int[] nums = {10, 20, 30};
change(nums);
System.out.println(nums[0] + ",␣" + nums[1]);
}
}
```

答案与深度解析 答案：`99, 20` 【核心认知框架】：考查数组引用传递机制。【执行流程深度拆解】：

- 传递：传递数组引用副本，指向同一堆对象。
- 修改：方法内修改堆内容，外部可见。

fifine专升本

- 结果：nums[0] 变为 99，nums[1] 不变。

3. 写出下列程序的输出结果：

```
class Father {
int x = 10;
public void show() { System.out.println("Father␣Show"); }
}
class Son extends Father {
int x = 20;
public void show() { System.out.println("Son␣Show"); }
}
public class Test3 {
public static void main(String[] args) {
Father f = new Son();
System.out.println(f.x);
f.show();
}
}
```

fifine专升本

答案与深度解析 答案：`10`
`Son Show` 【核心认知框架】：考查多态与方法/字段绑定区别。【执行流程深度拆解】：

- 字段：非多态，看左边（父类），输出 10。
- 方法：多态，看右边（子类），输出 Son Show。
- 字节码：字段访问 `getfield` 静态绑定，方法调用 `invokevirtual` 动态绑定。

4. 写出下列程序的输出结果：

```
public class Test4 {
public static void main(String[] args) {
String s1 = new String("Java");
String s2 = new String("Java");
System.out.println(s1 == s2);
System.out.println(s1.equals(s2));
}
}
```

答案与深度解析 答案：`false`
`true` 【核心认知框架】：考查字符串池与 equals 重写。【执行流程深度拆解】：
- **new String**：每次在堆上新建对象，地址不同，== 为 false。
- **equals**：String 重写为比较内容，内容相同为 true。

5. 写出下列程序的输出结果：

```
public class Test5 {
public static int testException() {
int res = 1;
try { res = 10 / 0; return res; }
catch (ArithmeticException e) { res = 2; return res; }
finally { res = 3; System.out.print("Finally␣"); }
}
public static void main(String[] args) {
System.out.println(testException());
}
}
```

答案与深度解析 答案：`Finally 2` 【核心认知框架】：考查 finally 与 return 执行顺序。【执行流程深度拆解】：
- 流程：try 异常 → catch 捕获 → res=2 → 准备返回 → 执行 finally → 输出 Finally → 返回 catch 中暂存的 2。
- 陷阱：finally 中修改 res 不影响返回值（基本类型），除非 finally 中有 return。

fifine专升本

五、编程题深度解析与设计模式分析（共 2 小题）

1. 面向对象基础设计（银行账户类）

答案与深度解析 核心设计思路：封装性 + 业务逻辑校验

【核心认知框架】

- 题目本质：考查类的基本结构、封装性及方法逻辑。
- 关键区分点：私有属性、构造器初始化、业务规则（余额检查）。
- 命题人意图：测试 POJO 设计规范及基本逻辑控制。

【关键技术点深度拆解】

- 封装性：
  - 属性 `private`，防止外部直接修改余额。
  - 通过 `deposit/withdraw` 方法控制状态变更。
- 构造方法：
  - 使用 `this` 区分参数与成员变量。
  - 确保对象创建时状态合法。
- 业务逻辑：
  - 取款前检查 `balance >= amount`。
  - 体现面向对象的行为建模。

【参考代码核心片段】

```java
public void withdraw(double amount) {
    if (balance >= amount) {
        balance -= amount;
        System.out.println("取款成功");
    } else {
        System.out.println("余额不足");
    }
}
```

2. 接口与多态的综合应用（图形面积计算）

答案与深度解析 核心设计思路：接口隔离 + 多态数组

【核心认知框架】

- 题目本质：考查接口定义、实现及多态调用。
- 关键区分点：接口引用指向实现类对象，统一调用 `getArea()`。
- 命题人意图：测试多态性在解耦中的实际应用。

fifine专升本

【关键技术点深度拆解】

- 接口定义：
  - Shape 定义行为契约 getArea()。
  - 符合依赖倒置原则（DIP）。
- 多态数组：
  - Shape[] shapes 可存放任何实现类。
  - 循环调用 shapes[i].getArea() 动态绑定。
- 扩展性：
  - 新增图形只需实现接口，无需修改测试代码。

【参考代码核心片段】

```
Shape[] shapes = new Shape[2];
shapes[0] = new Rectangle(4, 5);
shapes[1] = new Circle(3);
// 多态调用
for (Shape s : shapes) {
    totalArea += s.getArea();
}
```

fifine专升本

——讲义结束，祝学习顺利——